

ANDHRA PRADESH CENTRE FOR FINANCIAL SYSTEMS AND SERVICES

(APCFSS)

Internal Note

Integration of Nidhi ESS AI Agent

with Budget (CFMS Ledger) AI Agent

A join of the people ledger and the money ledger — so the State can answer, in one place, whether this month’s salary can be paid, where an employee’s bill is stuck, and whether GPF / APGLI deductions actually reached Public Account.

To	Manager / Delivery Head, APCFSS
From	Nidhi ESS AI & Budget AI product teams
Date	19 September 2026
Subject	Proposed feature: Nidhi–Budget AI join (read-only pilot)
Classification	Internal — for discussion · Not for circulation outside APCFSS
Decision sought	Approve a 4-week, one-DDO, read-only pilot

THE IDEA IN FOUR SENTENCES

Nidhi already knows **who must be paid** and what was cut from the slip. Budget AI already knows **whether the HOA can pay** and whether that cut reached Public Account. Today those two facts never meet. This note proposes that they meet — without rewriting either product, without posting to SAP, and without showing DDO budgets to employees.

1. Purpose of this note

This note explains a proposed feature that connects two AI products already built inside APCFSS. It is written so that a manager can take a decision without reading architecture papers. The technical approach is included only to show that the two existing stacks can talk to each other without a rewrite.

It answers five questions: what we already have; what problem we are solving; what the feature looks like for an employee and a DDO; how Spring AI (Nidhi) and LangGraph (Budget) will integrate; and what we are asking approval for.

2. What we already have (do not rebuild)

APCFSS already runs two live AI agents on top of systems we own. They are complementary. They are not competitors.

Product	Stack	Already answers	Does not answer
Nidhi ESS AI nidhi.apcfss.in	Java Spring AI	Employee HR, salary, payslip, arrears, pension, GPF, APGLI	Whether HOA 010 can fund this month; where the CFMS bill is sitting; whether the GPF cut reached Public Account

Product	Stack	Already answers	Does not answer
Budget / CFMS Ledger AI	Python LangGraph	Budget, expenditure, receipts, Public Account, PD accounts, pending bills	Who must be paid this month; headcount vs provision; whose GPF was cut on the slip

Table 1. Current product ownership — people ledger vs money ledger.

In short: Nidhi is the **people ledger**. Budget AI is the **money ledger**. CFMS was always meant to be a single source of truth across establishment and accounts. SAP stores both, but no user gets one sentence from both. That sentence is the feature.

3. The problem we see every month

Salary day, GPF credits and pending bills already generate SRTS tickets and DDO escalations. The data to answer them exists. It lives in two places.

Who	What they see today	What is actually true
Employee	Opens Nidhi. No September payslip. Assumes payroll is broken. Calls DDO / SRTS / treasury.	The salary bill is pending at STO. Nidhi is correct (no slip). The status is on the Budget / CFMS side.
DDO / HoO	Submits salary bill. CFMS rejects it on Budget Availability Check, or parks it. Then reconciles 010 vs Nidhi strength in Excel.	DA arrear + new joinings + EL encashment already sat in Nidhi as liability, and 010 was short before submit.
Finance / HOD	Budget AI shows HOA 010 spent 112%. Instruction: “control expenditure.”	Nidhi has 22 contract regularizations. Payroll moved to 010; budget still assumed old strength. Cause is establishment, not waste.
GPF / APGLI subscriber	Payslip shows the cut. Passbook / policy does not move. Blames Nidhi or the DDO.	Deduction happened in HCM. Remittance to Public Account did not. Only Budget AI can see that break.

Table 2. Same incident, two half-answers — that is the gap.

The CAG Information Systems Audit of CFMS has already documented the ugly form of this gap: excess pension drawals, duplicate payments, PD balances that administrators did not know about, and books that do not match. A monthly join of Nidhi and the ledger will not close every audit para. It will catch the recurring ones **in the month**, instead of three years later.

4. Proposed feature — three skills, not a new portal

We will not launch a third chatbot and we will not merge the two user interfaces. We will add three skills that **call the other product as a tool**. Each skill has a clear user, a clear question, and a clear privacy line.

4.1 Skill A — “Where is my salary?” (employee, on Nidhi)

Who: Government employee or pensioner, already logged into Nidhi ESS AI.
Trigger: No payslip this month, or “where is my salary / arrear / pension?”
What Nidhi does: Confirms there is no slip, then calls Budget AI with that CFMS ID and month only.
What the employee sees:

“Your September salary is in bill **2026-010-77821**. It is pending at STO Vijayawada since 15 September (4 days). This is not a payroll error. The payslip will appear after payment.”

Privacy line: the employee sees only *their* bill status. They never see DDO budget, HOA balances, or anyone else’s name. **Why it is useful:** this is the most common SRTS pattern. Nidhi stops being a dead payslip store and

becomes the place that tells the truth.

4.2 Skill B — “Will this month’s salary bill pass?” (DDO, on Budget AI)

Who: DDO / Head of Office, on Budget AI.

Trigger: Before submitting the monthly salary bill, or when BAC is about to fail.

What Budget AI does: Calls Nidhi for this DDO, this month: regular salary + arrear + pension (+ EL encashment if available) and headcount. Compares with HOA 010 remaining and pending salary bills already in CFMS.

What the DDO sees:

Salary cover — DDO XXXX — September 2026

Liability from Nidhi: Rs.1.12 crore · HOA 010 left: Rs.0.95 crore · Already pending salary bills: Rs.0.08 crore

Short by Rs.0.25 crore. Main cause: DA arrear batch for 40 people (Rs.18 lakh) + 6 new joinings.

Action: stagger arrears, seek reappropriation, or hold EL — *before* the bill is submitted.

Privacy line: DDO sees only that DDO. Finance / HOD may see department roll-up. **Why it is useful:** it prevents a failed salary bill instead of explaining it after 400 people miss credit. Salary day is a district-level problem. This is the one screen a DDO will open every month without training.

4.3 Skill C — “GPF / APGLI cut vs remittance” (DDO / treasury)

Nidhi already has GPF and APGLI on the employee side. Budget AI already has Public Account. Neither currently proves the other. Per DDO, per month, we compare: sum of GPF (and APGLI) deductions on Nidhi slips versus Public Account credit / remittance versus any related pending bill.

This DDO, August: GPF cut on 86 slips = Rs.8.4 lakh. Public Account credit = Rs.8.1 lakh. **Exception: 4 CFMS IDs — deducted on slip, not remitted.** Same pattern for APGLI premium and loan recovery.

What we do not send: GPF passbooks, APGLI policy PDFs, or full payslips into Budget AI. Only month + DDO + amounts + exception CFMS IDs. **Why it is useful:** money has already been taken from the employee. Today that becomes a grievance. After this, it becomes a reconciliation list. AG GPF and APGLI corporation disputes shrink.

Two further checks use the same join and need no new screens: **arrear cover** (Nidhi arrear queue vs 010 remaining) and **pension double-draw** (same CFMS ID on salary and pension in the same month, both paid in expenditure). CAG has already flagged excess pension. This is how we catch it in-month.

5. How this is useful — by stakeholder

Usefulness is not “we added AI.” Usefulness is a question the State cannot answer today in one place, answered in one card.

Stakeholder	What changes	Why they will care
Employee / pensioner	Knows where salary, arrear or pension is stuck, inside Nidhi.	Stops panic and the largest class of SRTS tickets.
DDO / HoO	Knows before submit whether 010 will hold, and why.	Salary day is their accountability.
Treasury / STO	Sees whether a pending bill is “no cover” or process lag.	Workload and fewer avoidable returns.

Stakeholder	What changes	Why they will care
HOD / Finance	Sees the cause of 010 overrun: strength, arrear, CER — not a slogan to “control spend.”	Fair budget control; evidence for reappropriation.
AG / APGLI / CAG	Exception lists instead of forensic reconstruction after three years.	Live control against known audit paras.
APCFSS	Two agents become one G2E × G2G product, without a new portal.	We stop being two chatbot demos and start being a control system.

Table 3. Who gains, and why the feature will be used.

THE ONLY THREE QUESTIONS THAT MATTER

- Three questions this join exists to answer:
1. Can we pay these people this month?
 2. If not, is it budget, a pending bill, or a wrong establishment?
 3. Money cut from the slip — did it actually reach GPF / APGLI / pension?

6. What this feature is not (out of scope)

Writing these down protects the pilot from becoming a platform programme.

Out of scope	Why
A new citizen / employee app or a third chatbot	Both user groups already have a place to talk.
Putting payslips inside Budget AI, or HOA balances inside employee Nidhi chat	Wrong audience; DPDP and role risk.
Agent posting or approving bills in SAP / CFMS	Human submits. Agent only shows and pre-checks.
Sharing Aadhaar, PAN, bank account, or full payslip across agents	Not required for the join. Totals + exception IDs are enough.
Rewriting Nidhi in Python or Budget AI in Java	Two products, two runtimes, one contract — same as CFMS talking to NIDHI today.
Receipts, PD radar, Works, GST, vendor “when will I be paid?”	Valid later products. They do not need Nidhi HR data. Keep them out of this pilot.
Training a new LLM or buying GPU	Existing agents plus two MCP servers are enough.

Table 4. Explicit non-goals for the pilot.

7. How we integrate Nidhi and Budget AI — MCP, not a rewrite

Nidhi ESS AI is **Spring AI (Java)**. Budget AI is **Python LangGraph**. We will not merge codebases. We will not share a JVM or a Python environment. We will expose each product as an **MCP server** (Model Context Protocol) and let each agent act as an **MCP client** of the other. MCP is the standard tool protocol Spring AI already ships (server + client starters) and that LangGraph consumes via **langchain-mcp-adapters**. Stdio MCP is for a laptop. We will use **Stateless Streamable HTTP** — Spring AI’s production default — on the AP data centre network, behind existing SSO.

INTEGRATION IN ONE PARAGRAPH

Employee talks only to Nidhi ESS AI. That agent is an MCP client of *nidhi-ess-mcp* (own data) and of *cfms-ledger-mcp* for *one* tool: bill status.

DDO / FD talks only to Budget AI. That agent is an MCP client of *cfms-ledger-mcp* (own data) and of *nidhi-ess-mcp* for *one* tool: DDO-month liability.

The user never switches bot. The other system is a tool, not a chat.

7.1 What MCP servers we need to build (only two)

Do not build a third “join” MCP. Salary-cover and GPF-break are **agent skills** that call tools from both servers. Do not wrap SAP as a generic SQL MCP. Every tool is a named use-case with a role check.

MCP server	Runtime	Talks to	Owner
nidhi-ess-mcp	Java / Spring AI spring-ai-starter-mcp-server-webmvc protocol = STATELESS	NIDHI APIs / DB only (salary, payslip, GPF, APGLI, pension, HR)	Nidhi ESS AI team
cfms-ledger-mcp	Python FastMCP / official MCP SDK Streamable HTTP, stateless	CFMS read replica / existing Budget AI data layer (budget, exp, PA, PD, pending bills)	Budget AI team

Table 5. Two MCP servers — one per ledger. Nothing else for the pilot.

Transport: HTTPS on internal URLs, for example `nidhi-mcp.apcfss.internal/mcp` and `cfms-mcp.apcfss.internal/mcp`. **Auth:** same SSO bearer the user already has; the MCP server enforces role. **Allow-list:** each client is configured with only the tools it is allowed to see. Nidhi agent must not discover HOA-balance tools. Budget agent must not discover payslip tools.

7.2 Tools on nidhi-ess-mcp

Publish only what the other agent may call. Employee payslip / GPF passbook stay as **local Spring AI tools** inside the Nidhi agent — they are not on the MCP surface that Budget AI can see.

Tool (MCP name)	Who may call	In / out	Pilot?
nidhi_get_liability	DDO / HoO / FD (Budget AI client)	In: ddo, fy, month Out: regular, arrear, pension, gpf_deducted, apgli_deducted, headcount, exception CFMS IDs	Yes — Skill B
nidhi_get_gpf_deductions	DDO / treasury	In: ddo, month Out: total deducted, exception IDs (no passbooks)	Stretch — Skill C
nidhi_get_apgli_deductions	DDO / treasury	Same shape as GPF	Stretch — Skill C
payslip / gpf_balance / pension_slip	Employee, Nidhi agent only — not published on MCP to Budget	Own CFMS ID	Already built; do not MCP-share

Table 6. *nidhi-ess-mcp* tool catalogue.

7.3 Tools on cfms-ledger-mcp

Tool (MCP name)	Who may call	In / out	Pilot?
cfms_get_bill_status	Employee (Nidhi agent client)	In: cfms_id, month Out: bill_no, type (salary / arrear / pension), stage, days pending	Yes — Skill A
cfms_get_hoa_balance	DDO / FD (Budget AI, local)	In: ddo, hoa, fy Out: BE, spent, pending, remaining (010 first)	Yes — Skill B
cfms_get_pending_bills	DDO / FD	In: ddo, month, bill_type Out: count, amount, stages	Yes — Skill B
cfms_get_pa_credits	DDO / treasury	In: ddo, month, head (GPF / APGLI) Out: remitted amount	Stretch — Skill C
PD / receipts / works tools	Budget AI local only — not published to Nidhi client	Already in Budget agent	No — out of this join

Table 7. cfms-ledger-mcp tool catalogue.

7.4 How each existing agent is wired

Nidhi ESS AI (Spring AI) — MCP client + MCP server. Keep the current ChatClient. Add spring-ai-starter-mcp-server-webmvc and expose nidhi_get_liability as an @McpTool, protocol STATELESS. Add spring-ai-starter-mcp-client with one Streamable-HTTP connection to cfms-ledger-mcp. Bind only **cfms_get_bill_status** into the employee ChatClient tool list. When the user has no payslip, the model calls that one remote tool. It cannot see HOA balances because those tools are not in its list.

Budget AI (LangGraph) — MCP client; Python MCP server beside it. Keep the current graph. Stand up cfms-ledger-mcp in the same Python service (FastMCP) wrapping the data functions you already use for budget, pending bills and Public Account. In the graph, use MultiServerMCPClient (langchain-mcp-adapters) with two connections: local cfms-ledger-mcp and remote nidhi-ess-mcp. Skill B node calls nidhi_get_liability + cfms_get_hoa_balance + cfms_get_pending_bills, then formats the cover card. Do not give the Budget graph the employee payslip tools.

Agent	MCP client of	Tools allowed on that client
Nidhi ESS AI (employee chat)	cfms-ledger-mcp	cfms_get_bill_status only
Nidhi ESS AI (own domain)	local Spring @Tool (not remote MCP)	payslip, GPF, APGLI, pension — already built
Budget AI (DDO chat)	nidhi-ess-mcp	nidhi_get_liability; later GPF/APGLI deductions
Budget AI (own domain)	cfms-ledger-mcp (same process)	hoa_balance, pending_bills, pa_credits, existing ledger tools

Table 8. Allow-list — this is the security design, not an afterthought.

7.5 What we will not build as MCP

Do not build	Why
A third “join” or “salary_fit” MCP server	That is an agent skill composing two tools. A third server becomes another system of record.
Generic SAP / HANA SQL MCP	The model must not invent queries. Named tools only. Protect HANA.

Do not build	Why
Stdio / desktop MCP (Claude Desktop style)	Wrong runtime. These are two HTTP services on the data centre, not a laptop.
Write tools (post bill, approve leave, change HOA)	Pilot is read-only. Human submits in CFMS.
MCP tools that return payslip PDF, Aadhaar, bank, PAN	DPDP. Aggregates and exception IDs only across the boundary.
Shared vector DB as “integration”	Memory is not the join. Live MCP tools are the join.

Table 9. MCP anti-catalogue.

7.6 Contract still in force (join keys, audit, memory)

Join keys: CFMS ID, DDO code, HOA (010 / 040 first), bill number, FY, month. **Audit:** every MCP call logged (agent, user role, tool name, keys, latency; no payslip body). Same retention as CFMS. **Memory:** Redis session may hold role, DDO, FY. Liability and balances always from MCP tools, never from chat history. **Cache:** DDO-month aggregates until payroll close — so chat traffic does not hit production HANA per token.

7.7 Ownership

Piece	Owner
nidhi-ess-mcp + Skill A (where is my salary)	Nidhi ESS AI team (Spring AI)
cfms-ledger-mcp + Skills B and C (cover, GPF break)	Budget AI team (LangGraph / FastMCP)
Tool allow-lists, SSO on MCP HTTP, audit format	Joint — one page, signed by both tech leads
Pilot DDO relationship and feedback	Delivery / CFMS functional

Table 10. Who owns which MCP — so the pilot does not stall.

8. Data sharing and privacy

This feature is useful only if it is boring from a privacy point of view. The rule is: **aggregates and exception IDs cross the boundary; personal ledgers do not.**

Nidhi has	Budget AI is allowed to receive
Full payslip, GPF passbook, APGLI policy, HR file	No
Aadhaar, PAN, bank account, mobile	No
Employee chat history	No
DDO-month totals for salary / arrear / pension / GPF / APGLI	Yes
Headcount and exception CFMS IDs	Yes
Bill number for “where is my salary”	Yes — that employee only, under their token

Table 11. Data that may and may not cross.

Role matrix: an employee asking Budget AI “how much salary budget is left for my DDO?” is refused. That is a DDO question. Budget AI never writes into payroll, e-SR, leave or GPF. Nidhi never learns HOA, PD or receipts. DPDP: the employee can be shown what was retrieved about them (bill status). DDO cards are official records, same retention as finance (suggested 8 years, aligned to CFMS).

9. Proposed pilot (four weeks, one DDO)

We are not asking for a programme. We are asking for a time-boxed trial that can fail cheaply.

Week	Work	Exit check
Week 1	Freeze MCP tool schemas and allow-lists. Role matrix. Audit log format. Identify one DDO + one treasury. Read access only.	One-page contract signed by both tech leads.
Week 2	nidhi-ess-mcp and cfms-ledger-mcp live on non-prod (Stateless HTTP). No SAP write. Cache DDO-month aggregates until payroll close (protect HANA).	Budget AI can call nidhi_get_liability; Nidhi can call cfms_get_bill_status.
Week 3	Skill A on Nidhi (where is my salary). Skill B on Budget AI (salary_fit on 010 only).	DDO can open the cover card. One employee test ID gets bill status.
Week 4	DDO and treasury use it on live month. Capture SRTS and BAC numbers. Optional: GPF break on the same DDO if Skill B is stable.	Review with manager: keep, iterate, or stop.

Table 12. Four-week pilot — salary 010 first, GPF only if salary cover works.

In: one DDO, one month, HOA 010, read-only, Skills A and B. **Stretch if stable:** Skill C (GPF) on the same DDO. **Out:** PD, receipts, works, employee budget view, SAP write, second DDO, GPU, new LLM.

10. How we will know it worked

If these four do not move, the integration is only architecture and we should stop.

Metric	How measured	Direction we want
SRTS tickets “salary not credited” for the pilot DDO	SRTS category count, same month last year vs pilot month	Down
Salary bills rejected / returned on BAC for that DDO	CFMS bill log	Down (shortfall seen before submit)
Time for DDO to know 010 cover	DDO feedback: minutes vs current Excel	From hours / guesswork to one card
GPF “deducted, not remitted” exceptions open after T+7	Skill C exception list, if enabled	Towards zero

Table 13. Success metrics for the pilot review.

11. Risks and how we will handle them

Risk	Mitigation
Load on SAP HANA from chat traffic	DDO-month aggregate, cached until payroll close. No per-message SQL on production HANA. Read replica if available.
Wrong cover number on salary day	Card is advisory. Human submits the bill. We show source (Nidhi liability vs CFMS 010) so the DDO can disagree.
Privacy / DPDP / Aadhaar	No PII in Budget AI. Employee token scoped to own CFMS ID. Role check on every tool call.
Two languages, two teams, delay	Two MCP servers, HTTP. Contract in week 1. No shared runtime. Same pattern as CFMS talking to NIDHI.

Risk	Mitigation
Scope creep into PD, works, new portal	Table 4 is binding for the pilot. Any extra skill needs a new note.
User ignores the card	Pilot DDO nominated; we sit with them for one salary cycle. If they do not open it, we stop — not a training campaign.

Table 14. Risks the manager is likely to raise.

12. Decision requested

Approval is requested for the following, and only the following:

- 1. A **4-week pilot** on one DDO and one treasury, Skills A and B, HOA 010, read-only.
- 2. Permission to stand up **two internal read-only MCP servers** (nidhi-ess-mcp and cfms-ledger-mcp), using existing SSO, with audit logging.
- 3. Nomination of **one DDO and one treasury officer** who will actually use the cards and give feedback.
- 4. A **review meeting** at the end of week 4 against Table 13 — keep, iterate, or stop.

We are not requesting a new product name, a new budget head, GPU, a rewrite, or write access to SAP.

Annexure A — Thirty-second pitch (for the meeting)

SAY THIS FIRST — DO NOT OPEN WITH THE STACK

Sir, Nidhi already knows who must be paid and what was cut from the slip — salary, GPF, APGLI, pension. Budget AI already knows whether the HOA can pay and whether that cut reached Public Account. Today those two facts never meet, so employees raise SRTS tickets and DDOs fail salary bills. This is not a new chatbot. It is a join of the two products we already built. Nidhi stays in Spring AI. Budget stays in LangGraph. They call each other as MCP tools — two servers, read-only. We ask for one DDO, one month, read-only.

Annexure B — If asked “two technologies — is that a problem?”

No. Nidhi remains Spring AI. Budget AI remains LangGraph. Integration is **two MCP servers over Stateless HTTP**, not a shared codebase. Spring AI already has MCP server and client starters. LangGraph uses langchain-mcp-adapters. We already integrate Java and SAP ABAP in CFMS / NIDHI. This is the same idea: Nidhi exposes nidhi_get_liability; Budget exposes cfms_get_bill_status. Both read-only.

Annexure C — One-line versions by audience

Audience	One line
Finance / CFMS	Before salary day, the DDO will know if HOA 010 can cover Nidhi’s actual liability — not last year’s strength.
Nidhi / HCM	When there is no payslip, Nidhi will tell the employee which CFMS bill is pending, instead of sending them to SRTS.
Delivery / tech	Two existing agents, two MCP servers, no rewrite, no SAP write, four-week pilot.
CEO / Board	People ledger and money ledger finally answer the same question: can we pay these people this month, and did the slip-cut reach GPF.

Table 15. Open with the line that matches who is in the room.

Annexure D — Glossary (for non-CFMS readers)

Term	Meaning in this note
Nidhi	APCFSS employee portal (nidhi.apcfss.in): payslip, ESS, GPF, APGLI, pension, HR.
CFMS	Comprehensive Financial Management System — SAP S/4HANA public finance of GoAP, run by APCFSS.
DDO	Drawing and Disbursing Officer — submits bills for an office.
HOA 010 / 040 / 020	Heads of Account: salaries / pensionary charges / wages.
BAC	Budget Availability Check — CFMS will not pass a bill if the HOA has no cover.
Public Account	Government as banker (GPF and similar). Not the Consolidated Fund. Budget AI already queries it.
PD account	Personal Deposit account. In Budget AI already. Not part of this Nidhi join.
SRTS	APCFSS support / ticketing for CFMS and Nidhi users.
Spring AI	Java framework used by Nidhi ESS AI. Also hosts nidhi-ess-mcp.
LangGraph	Python agent framework used by Budget AI.
MCP	Model Context Protocol — standard way an agent discovers and calls tools on another service over HTTP.
nidhi-ess-mcp	MCP server we will build on Spring AI. Exposes DDO-month liability (not payslips) to Budget AI.
cfms-ledger-mcp	MCP server we will build in Python. Exposes bill status to Nidhi and HOA/pending/PA tools to Budget AI.

Table 16. Terms used in this note.

Prepared for internal discussion at APCFSS, Mangalagiri. This note does not commit Finance Department, DTA or any DDO until the pilot is approved. Figures in the sample cards (Rs.1.12 crore, bill 77821, 86 slips) are illustrations, not live data.

Contact for this note: Nidhi ESS AI team and Budget AI team, APCFSS.